AutoGrader

A Rule-Driven Evaluator for C Programs using open weight LLM-Models and modularized for expert analyzers on different aspects of evaluation metrics for independent and consistent evaluation.

Balaji Vara Prasad Dega BALAJI_2611AI05@IITP.AC.IN
2611AI05

Uddipta Arun Paul UDDIPTA_2611AI06@IITP.AC.IN
2611AI06

Suva Pratyush Tripathy SUVA_2611AI14@IITP.AC.IN
2611AI14

Asadhi Vamshi Raj ASADHI_2611MC14@IITP.AC.IN
2611MC14

Abstract

Manual grading of C programs is slow, inconsistent, and hard to scale. This report presents **AutoGrader**, an agentic, rule-driven system that evaluates student C submissions unattended and returns one reproducible grade with actionable feedback. Given a submission and an instructor-authored rules file, the system compiles and runs the program in a sandbox with fixed time and memory limits, evaluates it against every metric, and produces both a JSON report and a student-facing text report.

A coordinator agent first performs a sanity sweep (compile check, smoke run, static checks). If the program crashes or never halts, execution-based analyzers are skipped and a partial report is emitted. Otherwise, independent analyzers assess correctness, compiler warnings, structure, modularity, edge cases, empirical time complexity, memory safety (ASan/UBSan), readability (a local LLM via Ollama[1] or any other open-source/open-weight LLMs that are more compatible with the local system configuration and capability), and originality (similarity scoring). Metrics, thresholds, and weights are stored in an external JSON file, so instructors can change them without editing code.

Reproducibility is ensured through fixed test sets, hashed rules, pinned tool versions, and LLM calls with temperature zero and a fixed seed. For incorrect submissions, the agent generates counter-test cases covering corner cases such as overflow, reporting the failing input, expected output, and an explanation. The report also discusses real-life usefulness and concrete limitations of the system.

GitHub code Repository: https://github.com/BALAJI24092001/AutoGrader

1. Progress - Week 3

The repository structure has been created with directories for analyzers, rules, submissions, tests, documentation and reports. Sample submissions and test cases have been added. The following initial components are available:

- A program runner with time, memory and output limits.
- Analyzers for compiler warnings, static analysis and sanitizer runs.
- An edge-case runner that compares a submission with a reference solution using fixed and seeded random inputs for the example problem.
- Search problem test inputs, expected outputs and edge cases.
- A C code similarity analyzer on a separate branch.

These components still need integration and testing as a complete grading system. Further isolation and security testing are needed before running untrusted submissions.

1.1 Work Completed by Members

The following is based on the repository branches and visible commits.

- **Balaji Vara Prasad Dega:** Created the repository structure and rules skeleton, added sample submissions and merged contributions into the main branch (`balaji-analyzer`).
- **Uddipta Arun Paul:** Added the program runner and initial compiler warning, static analysis and sanitizer modules (`uap-ai`).
- **Suva Pratyush Tripathy:** Added search test cases and an edge-case runner (`pratyush-dev`).
- **Asadhi Vamshi Raj:** Added C code similarity analysis on `vamshi-mc`; this branch has not yet been merged into main.

1.2 Next Steps

Integrate the modules with a coordinator, complete rules and weighted scoring, test the system on different submissions, review the similarity module, and develop the planned language-model-based analysis and reports.

2. Distribution of Work Among Group Members

2.1 Work Allocation Plan

The project is divided into four parts. Each part has a clear owner and a defined interface (the JSON analyzer contract: score, evidence, feedback), so members can work in parallel and integrate later.

2.2 Coordination and Integration

- All analyzers follow one agreed interface: the input is the source file, the binary and the params slice, and the output is a score in $[0, 1]$, the evidence and a feedback string.
- Code is kept in a shared GitHub repository, with one folder per analyzer. Another member reviews each pull request before it is merged.

Member	Work
Balaji Vara Prasad Dega	Main controller, rules loading, safe running, final grade
Suva Pratyush Tripathy	Test runner, edge-case generator, complexity check
Uddipta Arun Paul	Compiler, structure and memory-safety checks
Asadhi Vamshi Raj	LLM review, similarity check, reports, README, video

- The group meets briefly twice a week, and Member 1 integrates the analyzers into the coordinator as they become ready.
- Every member reviews and can explain the whole codebase, since each member must own the code and the report.

3. Future Plan and Timeline

The first two weeks focus on learning the problem and the tools. The last two weeks focus on implementation, integration and testing.

Week	Phase	Planned activities
Week 1-2	Learning	Study the problem statement and the sample rules file. Learning and understanding the tools to be used for the project. Deciding the work split and work flow.
Week 3	Implementation	*Part 1:* sandbox, first pass, rules loader and coordinator skeleton. *Part 2:* `io_tests`, `edge_case_runner` and `sanitizer_run`. *Part 3:* `compiler_warnings`, `static_analysis` and `function_split`. *Part 4:* test sets, reference solutions and the `similarity_flag` analyzer.
Week 4	Integration and testing	*Part 1:* collation, JSON and text reports, parallel dispatch. *Part 2:* `complexity_estimator` and optional profiling. *Part 3:* `llm_review`. *Part 4:* counter-test generation, reproducibility tests on the sample submissions, limitations section, README, final report and video. *All members:* integrate, fix bugs and rehearse the demo.

Table 1: Four-week implementation timeline.